

WikiBridge 软件测试报告

项目名称：零代码内网穿透本地知识库平台 WikiBridge
文档类型：软件测试报告
参考模板：测试计划（GB8567-88）、测试分析报告（GB8567-88）
文档版本：v2.2
编写日期：2026 年 6 月 25 日
编写单位：WikiBridge 项目组（武汉大学软件工程课程设计项目组）

修订历史

版本	日期	主要变更
v1.0	2026-06-20	形成初版测试报告，覆盖黑盒、白盒和系统测试设计。
v1.1	2026-06-23	按 fQwQf/wikibridge 最新源码调整测试对象，补充 llm_wiki 、OpenCode 知识库模式、Docker Compose 集成栈和 BearFRP 自动发布链路。
v2.0	2026-06-25	按 GB8567-88《测试计划》和《测试分析报告》模板重组文档，补充测试计划、测试设计、执行结果、功能结论、分析摘要和资源消耗。
v2.1	2026-06-25	补充检查点，整合 Compose 当前运行态、BearFRP TCP 发布入口和桌面端知识库闭环测试结果。
v2.2	2026-06-25	将失败-修正-成功、失败-规范化-成功和失败-降级-成功过程样例纳入重点回归清单。

第一部分 测试计划

1 引言

1.1 编写目的

本文档用于说明 WikiBridge 软件系统的测试计划、测试设计、预期输出、评价准则以及测试分析结论。文档参考 GB8567-88 中《测试计划》和《测试分析报告》的结构编写，作为课程设计验收、项目组内部测试实施和后续缺陷修正的依据。

预期读者包括：

- 课程设计指导教师和验收人员；
- WikiBridge 项目开发与测试成员；
- 后续维护本项目的开发人员；
- 需要复现实验环境和演示流程的使用者。

1.2 背景

被测软件系统名称为 **WikiBridge 零代码内网穿透本地知识库平台**。系统面向“本地知识库生成、Web 消费端访问、零代码发布”的课程设计场景，目标是在用户不编写网络配置代码的前提下，将本地知识库通过统一 Web 入口、桌面端本地浏览能力和 BearFRP 内网穿透能力发布到可访问的 Web 地址。

项目采用多服务集成结构，核心组成如下：

- `llm_wiki`：知识库生成、检索、图谱、评审和 API 服务端；
- `opencode`：知识库消费端，启用 Knowledge Base 模式后提供私有知识库和公开 Wiki 的隔离访问；
- `nginx`：统一入口网关，转发浏览器请求到 OpenCode Web 服务；
- `bearfrp`：用户、HTTP/TCP 代理、frps 插件和发布脚本控制面；
- `bearfrp-wikibridge-frpc`：自动发布 sidecar，用于演示环境中自动注册用户、创建代理并启动 frpc；
- `desktop-frp`：桌面端项目管理、Compile/Link、本地 mask 发布页和素材浏览入口；
- `docker-compose.yml`：系统一键部署和服务编排入口。

测试计划开始前应完成以下工作：

1. 最新源码已同步，参考基线为 GitHub 仓库 <https://github.com/fQwQf/wikibridge> 主分支。
2. Docker、Python、Node.js/npm、Bun、Rust/Tauri 等基础环境已准备。
3. `.env` 或等价环境变量已按部署要求配置。
4. 测试人员已了解 Docker Compose、HTTP API、frp/frpc/frps 和基础 Web 操作。

1.3 定义

术语	定义
WikiBridge	本项目名称，指由知识库服务、消费端和内网穿透发布层组成的集成系统。
LLM Wiki	<code>llm_wiki</code> 子系统，负责知识库项目、文件、搜索、图谱和评审等能力。
OpenCode KB 模式	OpenCode 的知识库模式，限制文件访问范围，只允许访问用户私有知识库和公开只读 Wiki。
BearFRP	本项目中的 frp 控制面，负责用户认证、流量额度、HTTP/TCP 代理创建和脚本生成。
frps	frp 服务端，用于接收 frpc 连接并提供 HTTP vhost 发布入口。
frpc	frp 客户端，用于把内网服务连接到 frps。
sidecar	与主服务并行运行的辅助容器，本项目中指自动创建并启动 WikiBridge frpc 的容器。
KB Guard	OpenCode KB 模式中的路径权限守卫模块。
桌面端知识库闭环	桌面端创建项目、Compile 生成 Wiki、Link 生成链接报告，并通过本地 mask 服务浏览 Wiki 的完整流程。
E2E	End-to-End，端到端测试。
Smoke Test	冒烟测试，用于快速确认核心服务是否可用。

1.4 参考资料

序号	文件或资料	来源
1	《测试计划（GB8567-88）》	/root/whu-os/doc/测试计划（GB8567—88）.pdf
2	《测试分析报告（GB8567-88）》	/root/whu-os/doc/测试分析报告（GB8567—88）.pdf
3	WikiBridge 最新源码	https://github.com/fQwQf/wikibridge
4	WikiBridge 部署说明	README.deploy.md
5	Docker Compose 配置	docker-compose.yml

序号	文件或资料	来源
6	OpenCode KB 模式说明	opencode/doc/kb-mode.md
7	BearFRP 后端源码	bearfrp/backend
8	LLM Wiki API server 源码	llm_wiki/src-tauri/src/api_server.rs
9	BearFRP 桌面端源码	desktop-frp 、 bearfrp/desktop-frp

2 计划

2.1 软件说明

软件部件	主要功能	输入	输出	质量指标
Docker Compose 编排	启动和连接全部服务	.env、镜像构建上下文、端口配置	运行中的容器、网络、数据卷	服务依赖正确，健康检查可用，端口映射清晰。
nginx 网关	提供统一 HTTP 入口	浏览器或 curl 请求	OpenCode Web 响应、API 代理响应	WebSocket 支持、50MB 上传限制、上游转发稳定。
LLM Wiki 服务	项目、文件、搜索、图谱、评审、重扫	项目目录、API 请求、Token	JSON API 响应、检索结果、图谱数据	鉴权正确、限流有效、文件大小和请求体边界受控。
OpenCode KB 模式	知识库消费和访问控制	用户请求、模型工具调用、文件路径	Web UI、文件列表、工具执行结果	私有目录可读写，公开 Wiki 只读，项目外路径拒绝。
BearFRP 控制面	用户、余额、HTTP/TCP 代理和脚本管理	注册登录信息、代理配置、管理员配置	代理记录、发布 URL、frpc 脚本	鉴权可靠、端口或子域名分配正确，错误提示明确。
frps 插件	校验 frpc 连接和 HTTP 代理	frps plugin 回调、token、代理名、子域名	允许或拒绝连接	旧 token、错子域名拒绝。
自动发布 sidecar	自动创建 WikiBridge 发布代理	BearFRP API、环境变量、frpc 版本	frpc.toml 、发布 URL、frpc 进程	幂等创建，错误可定位，发布入口指向 nginx。
桌面端知识库闭环	项目管理、Compile、Link、本地 mask 发布	本地素材、Wiki 页面、前端操作	Wiki 页面、链接检查报告、本地浏览页	目录结构清晰，生成结果可浏览，错误提示可定位。

2.2 测试内容

测试标识符	测试名称	测试内容	测试目的	计划阶段
T-01	部署与启动测试	Docker Compose、健康检查、数据卷、网络和端口映射	验证系统能按部署说明启动并形成单入口服务	组装测试
T-02	LLM Wiki API 测试	/api/v1/health、projects、files、content、reviews、search、graph、rescan	验证知识库服务端接口和边界条件	组装测试
T-03	OpenCode KB 权限测试	私有目录、公开 Wiki、路径穿越、软链接、shell/pty/mcp/config 禁用	验证知识库模式访问控制	组装测试
T-04	BearFRP 用户与代理测试	注册、登录、充值、HTTP 代理创建、脚本、删除	验证 HTTP 内网穿透控制面核心业务	组装测试
T-05	frps 插件鉴权测试	Login、NewProxy、Ping、CloseProxy	验证 frpc 连接和代理参数校验	组装测试
T-06	自动发布 sidecar 测试	自动注册、余额补足、代理幂等创建、frpc 配置重写	验证一键演示发布链路	确认测试
T-07	端到端访问测试	nginx 入口、OpenCode 页面、LLM Wiki 桥接、BearFRP 发布 URL	验证用户可从浏览器完成访问闭环	确认测试
T-08	异常与安全测试	默认密码、错误 token、请求体过大、子域名冲突、基础 XSS 样例、越权访问	验证错误处理和安全边界	确认测试
T-09	桌面端知识库闭环测试	项目目录初始化、Compile 生成 Wiki、Link 生成报告、本地 mask 发布页浏览	验证桌面端从素材到 Wiki 浏览的可演示闭环	确认测试

2.3 测试 T-01：部署与启动测试

本测试由项目组测试成员执行，被测试部位包括 `docker-compose.yml`、`nginx.conf`、各服务 Dockerfile 和容器健康检查。

2.3.1 进度安排

日期	工作内容
2026-06-20	检查 <code>.env.example</code> 、端口配置、服务依赖和数据卷配置。
2026-06-20	执行 Compose 构建和启动，观察健康检查和日志。
2026-06-21	执行 nginx 单入口和基础健康检查，整理部署测试输出。

2.3.2 条件

类型	要求
设备	一台 Linux 测试主机，至少 4 核 CPU、8GB 内存、20GB 可用磁盘。
软件	Docker、Docker Compose、curl、bash。
人员	1 名测试执行人员，熟悉 Docker Compose 和 HTTP 请求。

2.3.3 测试资料

- `docker-compose.yml`
- `nginx.conf`
- `.env.example`
- `README.deploy.md`
- 容器日志和健康检查输出

2.3.4 测试培训

测试人员需掌握 `docker compose up --build -d`、`docker compose ps`、`docker compose logs`、`curl` 的基本使用方式，并了解各服务之间依赖关系。

2.4 测试 T-02：LLM Wiki API 测试

本测试由项目组测试成员执行，被测试部位包括 `llm_wiki/src-tauri/src/api_server.rs`、`api_state.rs` 和 `llm_wiki/src/lib` 相关功能。

2.4.1 进度安排

日期	工作内容
2026-06-21	检查 API 路由、Token 配置和请求体限制。
2026-06-21	执行 health、projects、files、search、graph、rescan 接口测试。

2.4.2 条件

类型	要求
设备	Linux 测试主机或 Compose 容器环境。
软件	curl、Node.js/npm、Rust 工具链。
人员	1 名熟悉 HTTP API 和 JSON 响应的测试人员。

2.4.3 测试资料

- LLM Wiki 示例项目数据；
- API Token；
- 搜索关键词 `example`、`wiki`；
- 边界请求体和越权路径样例。

2.4.4 测试培训

测试人员需了解 `/api/v1` 路由结构、Token 传递方式、JSON 响应格式和常见 HTTP 状态码含义。

2.5 测试 T-03：OpenCode KB 权限测试

本测试由项目组测试成员执行，被测试部位包括 `opencode/packages/opencode/src/kb/guard.ts`、OpenCode server handlers 和文件工具。

2.5.1 进度安排

日期	工作内容
2026-06-22	准备私有目录、公开 Wiki、其它用户目录和软链接样例。
2026-06-22	执行读写权限、路径穿越、命令禁用和 Web UI 根目录测试。

2.5.2 条件

类型	要求
设备	Linux 测试主机或 OpenCode 容器。
软件	Bun、curl、OpenCode KB 模式运行环境。
人员	1 名熟悉文件权限和 Web API 的测试人员。

2.5.3 测试资料

- `opencode/scripts/init-kb-data.sh`
- `data/users/default/README.md`
- `data/wiki/README.md`
- 路径穿越样例：`../package.json`
- 软链接逃逸样例

2.5.4 测试培训

测试人员需了解 KB 模式中“我的知识库”和“公开 Wiki”的差异，以及 shell、pty、mcp、config 接口在 KB 模式下的禁止策略。

2.6 测试 T-04：BearFRP 用户与代理测试

本测试由项目组测试成员执行，被测试部位包括 `bearfrp/backend/routes/user_api.py`、`auth.py`、`port_pool.py`、`script_renderer.py`。

2.6.1 进度安排

日期	工作内容
2026-06-23	检查注册、登录、充值和代理创建接口。
2026-06-23	执行 HTTP 代理创建、脚本获取、重复子域名、删除和鉴权测试。

2.6.2 条件

类型	要求
设备	Linux 测试主机或 BearFRP 容器。
软件	Python、FastAPI 运行环境、curl、frp/frpc。
人员	1 名熟悉 HTTP API 和 frp 配置的测试人员。

2.6.3 测试资料

- 用户名：`wikibridge_test`
- 密码：`wikibridge-change-me`
- HTTP 子域名：`wiki-demo`
- HTTP vhost 端口：由 `BEARFRP_HTTP_VHOST_PORT` 配置

2.6.4 测试培训

测试人员需掌握 BearFRP 用户接口、HTTP 代理配置、frpc 配置结构和子域名唯一性校验逻辑。

2.7 测试 T-05：frps 插件鉴权测试

本测试由项目组测试成员执行，被测试部位为 `bearfrp/backend/plugin_handler.py`。

2.7.1 进度安排

日期	工作内容
2026-06-23	构造 Login、NewProxy、Ping、CloseProxy 请求，验证 token、代理名和子域名匹配逻辑。

2.7.2 条件

类型	要求
设备	BearFRP 后端运行环境。
软件	curl 或接口测试工具、frps 插件回调样例。
人员	1 名熟悉 frps 插件协议的测试人员。

2.7.3 测试资料

- 正确 frpc token；
- 旧 token；
- 错误 subdomain；
- 已删除代理 ID。

2.7.4 测试培训

测试人员需了解 frps plugin 的 Login、NewProxy、Ping、CloseProxy 时机和 BearFRP 后端状态字段。

2.8 测试 T-06：自动发布 sidecar 测试

本测试由项目组测试成员执行，被测试部位为 `docker/bearfrp-wikibridge-frpc/start.py`。

2.8.1 进度安排

日期	工作内容
2026-06-24	测试 sidecar 自动等待 BearFRP、注册/登录、充值、创建发布代理、改写 frpc 配置并启动 frpc。

2.8.2 条件

类型	要求
设备	Docker Compose 环境。
软件	Python sidecar、frpc、BearFRP API。
人员	1 名熟悉容器日志和 Python 脚本的测试人员。

2.8.3 测试资料

- WIKIBRIDGE_BEARFRP_USER
- WIKIBRIDGE_BEARFRP_PASSWORD
- WIKIBRIDGE_BEARFRP_PROXY_NAME
- WIKIBRIDGE_BEARFRP_PROXY_TYPE
- sidecar 日志输出

2.8.4 测试培训

测试人员需掌握 sidecar 环境变量、frpc.toml 结构和 BearFRP 发布 URL 判定方式。

2.9 测试 T-07：端到端访问测试

本测试由项目组测试成员执行，被测试部位为 nginx、OpenCode、LLM Wiki、BearFRP、frpc/frps 全链路。

2.9.1 进度安排

日期	工作内容
2026-06-24	从浏览器访问本地单入口和 BearFRP 发布 URL，执行项目列表、搜索、图谱和公开 Wiki 阅读流程。

2.9.2 条件

类型	要求
设备	Linux 服务端和至少一个浏览器客户端。
软件	Chrome/Edge/Firefox/Safari 之一，Docker Compose 全栈服务。
人员	1 名测试执行人员，1 名观察记录人员。

2.9.3 测试资料

- 浏览器访问地址：<http://localhost/>；
- BearFRP 发布 URL；
- 搜索关键词和示例 Wiki 页面。

2.9.4 测试培训

测试人员需了解用户演示路径：打开入口、浏览 Wiki、搜索、查看图谱、通过 BearFRP 发布 URL 访问。

2.10 测试 T-08：异常与安全测试

本测试由项目组测试成员执行，被测试部位为全部对外接口、网关、安全边界和默认配置。

2.10.1 进度安排

日期	工作内容
2026-06-25	执行默认凭据检查、错误 token、请求体过大、XSS、越权路径和子域名冲突测试，并整理最终报告。

2.10.2 条件

类型	要求
设备	Linux 测试主机。
软件	curl、浏览器、接口测试工具。
人员	1 名熟悉安全测试基础的测试人员。

2.10.3 测试资料

- 错误 token；
- XSS 样例：`<script>alert(1)</script>`；
- 超大请求体；
- 越权路径；
- 已占用子域名。

2.10.4 测试培训

测试人员需了解 XSS、路径穿越、默认密码、Token 鉴权和子域名冲突的基本判定方法。

2.11 测试 T-09：桌面端知识库闭环测试

本测试由项目组测试成员于 2026-06-25 执行，被测试部位包括桌面端项目管理、Compile / Link 命令、本地 mask 发布页和素材读取逻辑。

类型	要求或资料
环境	Linux 测试主机或桌面端开发环境；Rust/Tauri、Node.js/npm、浏览器。
测试资料	空项目、文本素材、图片素材、二进制素材、带 <code>[[...]]</code> 链接的 Wiki 页面样例。
人员要求	熟悉桌面端“创建项目、添加素材、Compile、Link、本地浏览”流程，以及 <code>raw/</code> 、 <code>wiki/</code> 、 <code>wiki/_link_report.md</code> 的含义。

3 测试设计说明

3.1 T-01 部署与启动测试设计

3.1.1 控制

测试采用半自动方式控制。测试人员通过命令行启动 Compose，通过 `docker compose ps`、`docker compose logs` 和 `curl` 记录结果。

3.1.2 输入

输入项	输入数据
环境变量	<code>.env.example</code> 默认值及测试环境覆盖值
启动命令	<code>docker compose up --build -d</code>
健康检查	<code>curl http://localhost/instance/llm-wiki/health</code> 、 <code>curl http://localhost:8000/api/show/online</code>

3.1.3 输出

预期输出：

- Compose 服务创建成功；
- nginx 单入口可响应；
- LLM Wiki health 返回 `ok=true` 或等价健康字段；
- BearFRP online 接口返回在线状态；
- 日志中无持续重启、端口绑定失败或依赖等待超时。

3.1.4 过程

1. 复制 `.env.example` 为 `.env`。
2. 执行 `docker compose up --build -d`。
3. 执行 `docker compose ps` 检查服务状态。
4. 执行 health 和 online 接口请求。
5. 记录容器状态、端口和接口输出。

3.2 T-02 LLM Wiki API 测试设计

3.2.1 控制

测试采用自动和半自动结合方式。普通接口使用 `curl`；前端和库测试使用 `npm run test:mocks`；Rust 测试使用 Cargo 测试入口。

3.2.2 输入

输入项	输入数据
health 请求	<code>GET /api/v1/health</code>
项目请求	<code>GET /api/v1/projects</code>
文件请求	<code>GET /api/v1/projects/default/files</code>
搜索请求	<code>POST /api/v1/projects/default/search</code> ，body 为 <code>{"query":"example"}</code>
图谱请求	<code>GET /api/v1/projects/default/graph</code>
错误 token	<code>Authorization: Bearer invalid-token</code>
超大请求体	大于 1MB 的 JSON body

3.2.3 输出

预期输出：

- health 接口返回服务状态、版本和鉴权配置；
- projects 返回项目数组；
- files/content 返回文件列表和内容；
- search 返回不超过 50 条搜索结果；
- graph 返回节点和边；
- 错误 token 返回 401；
- 超大请求体返回 400 或明确错误。

3.2.4 过程

1. 确认 LLM Wiki 服务启动。
2. 依次执行 health、projects、files、content、reviews、search、graph、rescan。
3. 构造错误 token 和超大 body。
4. 比较响应码、响应体和预期输出。

3.3 T-03 OpenCode KB 权限测试设计

3.3.1 控制

测试采用半自动方式。测试人员通过 Web UI、HTTP API 和 Bun 测试命令验证权限。

3.3.2 输入

输入项	输入数据
KB 模式环境变量	OPENCODE_KB_MODE=1
私有目录	data/users/default/README.md
公开 Wiki	data/wiki/README.md
越权路径	../package.json
其它用户目录	data/users/alice/README.md
禁用接口	/session/:id/shell、/pty、/mcp、/config

3.3.3 输出

预期输出：

- 私有目录读写成功；
- 公开 Wiki 读取成功、写入失败；
- 越权路径和其它用户目录访问失败；
- shell、pty、mcp、config 接口返回 403 或等价拒绝；
- Web UI 根节点只显示“我的知识库”和“公开 Wiki”。

3.3.4 过程

1. 执行 `bash scripts/init-kb-data.sh` 初始化数据。
2. 启动 OpenCode KB 模式。
3. 验证私有目录读写。

- 4. 验证公开 Wiki 只读。
- 5. 验证越权路径、软链接逃逸和禁用接口。
- 6. 执行 KB guard 单元测试。

3.4 T-04 BearFRP 用户与代理测试设计

3.4.1 控制

测试采用半自动接口测试方式。测试人员通过 curl 或接口测试工具控制注册、登录、充值、创建代理、获取脚本和删除代理。

3.4.2 输入

输入项	输入数据
用户名	wikibridge_test
密码	wikibridge-change-me
HTTP 代理	proxy_type=http, subdomain=wiki-demo
本地服务	local_ip=nginx 或 127.0.0.1, local_port=80
高级配置	HTTP basic auth、locations、host header rewrite
非法输入	空子域名、非法子域名、重复子域名
重复名称	两次提交相同 name

3.4.3 输出

预期输出：

- 注册和登录成功后返回用户公开信息；
- 充值后余额增加；
- HTTP 代理返回子域名 URL 和 frpc HTTP 配置；
- 重复名称、余额不足、非法子域名、重复子域名均返回 400；
- 删除代理后该子域名不可继续访问。

3.4.4 过程

- 1. 注册测试用户并登录。
- 2. 调用充值接口。
- 3. 创建 HTTP 代理并检查 frpc_config、subdomain URL 和脚本。
- 4. 执行重复名称、非法子域名、重复子域名和余额不足测试。
- 5. 删除代理并检查子域名释放或不可继续访问。

3.5 T-05 frps 插件鉴权测试设计

3.5.1 控制

测试采用接口模拟方式。测试人员构造 frps plugin JSON payload，发送到插件入口。

3.5.2 输入

输入项	输入数据
Login 正常	正确用户 token
Login 异常	旧 token 或伪造 token
NewProxy 正常	匹配的 proxy name 和 subdomain
NewProxy 异常	错误子域名、已删除代理
Ping	正确 token_version 和旧 token_version

3.5.3 输出

预期输出：

- 正常 Login/NewProxy/Ping 被允许；
- 旧 token、错子域名和已删除代理被拒绝；
- CloseProxy 后在线状态更新。

3.5.4 过程

1. 准备已创建代理和当前 token。
2. 构造 Login 请求。
3. 构造 NewProxy 请求。
4. 轮换 token 后再次构造 Ping。
5. 构造 CloseProxy 请求。
6. 检查代理在线状态和返回结果。

3.6 T-06 自动发布 sidecar 测试设计

3.6.1 控制

测试采用容器日志和配置文件检查方式。

3.6.2 输入

输入项	输入数据
API 地址	BEARFRP_API_URL=http://bearfrp:8000
用户	WIKIBRIDGE_BEARFRP_USER=wikibridge
代理名	WIKIBRIDGE_BEARFRP_PROXY_NAME=wikibridge
代理类型	http 或 tcp
本地服务	WIKIBRIDGE_BEARFRP_LOCAL_IP=nginx, WIKIBRIDGE_BEARFRP_LOCAL_PORT=80

3.6.3 输出

预期输出：

- sidecar 等待 BearFRP API 成功；
- 自动注册或登录成功；
- 余额不足时自动充值；

- 同名同类型代理被复用并更新；
- frpc.toml 中 server 地址被重写为 Compose 网络内地址，代理类型与配置一致；
- 日志或在线代理接口输出 WikiBridge 发布 URL。

3.6.4 过程

1. 启动 BearFRP、nginx 和 sidecar。
2. 查看 sidecar 日志。
3. 检查 /state/frpc.toml。
4. 在测试主机访问输出的 BearFRP 发布 URL。
5. 修改同名代理类型，验证错误提示。

3.7 T-07 端到端访问测试设计

3.7.1 控制

测试采用手工浏览器操作和 curl 辅助验证方式。

3.7.2 输入

输入项	输入数据
本地入口	http://localhost/
发布入口	BearFRP 发布 URL
搜索关键词	example、wiki
页面操作	打开项目、浏览文件、搜索、查看图谱

3.7.3 输出

预期输出：

- 本地入口打开 OpenCode KB 页面；
- 页面包含 KB mode meta；
- 项目和文件列表可见；
- 搜索结果可显示；
- 图谱节点和边可显示；
- BearFRP 发布 URL 与本地入口主要页面表现一致。

3.7.4 过程

1. 打开本地入口。
2. 检查 KB 模式页面。
3. 浏览公开 Wiki。
4. 执行搜索和图谱查看。
5. 打开 BearFRP 发布 URL。
6. 比较发布入口和本地入口表现。

3.8 T-08 异常与安全测试设计

3.8.1 控制

测试采用接口构造、浏览器观察和日志检查方式。

3.8.2 输入

输入项	输入数据
默认凭据	空密码、change-me 默认值
错误 token	Bearer invalid-token
基础 XSS 样例	<script>alert(1)</script>
越权路径	../package.json
大请求体	超过 1MB API body、超过 50MB 网关 body
子域名冲突	已被占用的 HTTP 子域名

3.8.3 输出

预期输出：

- 对外部署前默认凭据检查不通过并提示修改；
- 错误 token 返回 401；
- 基础脚本注入样例未触发弹窗或脚本执行；
- 越权路径被拒绝；
- 超大请求体被拒绝；
- 子域名冲突返回明确错误且不覆盖原代理。

3.8.4 过程

1. 检查 .env 中默认凭据。
2. 构造错误 token 请求。
3. 在 Markdown 或输入中加入基础 XSS 样例并浏览。
4. 构造路径穿越请求。
5. 构造超大请求体。
6. 构造子域名冲突代理创建请求。

3.9 T-09 桌面端知识库闭环测试设计

测试采用代码检查、桌面端操作和本地浏览页观察相结合的方式。测试人员通过桌面端触发项目创建、Compile 和 Link，随后通过本地 mask 发布页检查 Wiki 浏览结果。

输入项	输入数据
项目与素材	新建项目、空 raw/ 目录、文本素材、图片素材、二进制素材。
Wiki 链接	[[页面名]]、标题匹配、文件名匹配、slug 候选。
本地浏览路径	/、/api/project、/wiki/<page>、/raw/<asset>。

预期输出为：项目创建时建立 raw/ 和 wiki/ 子目录；Compile 生成 Wiki 页面；Link 生成 wiki/_link_report.md；本地 mask 发布页可浏览 Wiki 首页、单页文档和原始素材；[[...]] 链接和基础 Markdown 能正确渲染；成功提示和错误提示清晰。

1. 创建桌面端知识库项目，检查 `raw/` 和 `wiki/` 目录。
2. 在空项目上执行 `Compile`，检查 `wiki/README.md`。
3. 添加文本、图片和二进制素材后再次执行 `Compile`。
4. 执行 `Link`，检查 `wiki/_link_report.md`、反链统计和缺失链接记录。
5. 打开本地 `mask` 发布页 `/`、`/wiki/<page>` 和 `/raw/<asset>`。
6. 检查 Wiki 链接、基础 Markdown 渲染、成功提示和错误透传。

4 评价准则

4.1 范围

本次测试覆盖以下范围：

- Docker Compose 集成部署；
- nginx 单入口；
- LLM Wiki API；
- OpenCode KB 模式；
- BearFRP 用户、代理和脚本；
- frps 插件鉴权；
- 自动发布 `sidecar`；
- 浏览器端到端访问；
- 桌面端知识库 `Compile`、`Link` 和本地 `mask` 发布页；
- 关键异常和安全边界。

测试局限性：

- real-LLM 测试依赖外部模型 API Key，结果可能受网络和模型服务影响；
- BearFRP 发布测试依赖代理类型、`vhost` 或 `TCP` 端口、防火墙和 `subdomain host` 配置；
- 性能测试以课程设计演示负载为准，不覆盖企业级高并发场景；
- 移动端浏览器不作为本轮测试重点。

4.2 数据整理

测试数据按以下方式整理：

- 命令行输出整理为“命令、返回码、关键输出、判定结果”；
- HTTP 接口输出整理为“请求方法、URL、状态码、关键 JSON 字段、判定结果”；
- 浏览器测试整理为“入口地址、操作步骤、页面现象、判定结果”；
- 缺陷整理为“编号、现象、严重性、影响范围、处理建议、状态”；
- 自动化测试输出整理为“测试套件、用例数量、通过数量、失败数量、耗时”。

4.3 尺度

类别	通过尺度
功能正确性	核心接口状态码、返回字段和页面行为与预期一致。
稳定性	核心服务无持续重启，主要 E2E 流程可连续执行 3 次。
安全性	Token、路径隔离、默认凭据、基础 XSS 样例和 frps 插件鉴权均符合预期。
资源一致性	HTTP 子域名创建、重复校验、删除后的状态一致。
兼容性	Chrome、Edge、Firefox/Safari 至少各完成一组正交测试。

类别	通过尺度
性能边界	请求体、文件大小、并发槽和限流按源码约束生效。
缺陷准出	无阻塞级缺陷；高严重性风险均有规避措施或关闭记录。

第二部分 测试分析报告

1 引言

1.1 编写目的

本部分用于汇总 WikiBridge 测试工作的执行情况、测试结果、功能结论、缺陷限制、改进建议和资源消耗，为课程设计验收提供统一的质量评价依据。

1.2 背景

被测测试软件系统为 WikiBridge。任务提出者、开发者和测试者均为 WikiBridge 项目组；用户为课程设计验收人员和演示使用者。测试环境为 Linux + Docker Compose，与实际运行环境可能存在 subdomain host、HTTP vhost 端口、TCP 远程端口开放、浏览器版本和外部模型服务可用性差异。这些差异主要影响 BearFRP 发布 URL 可达性、real-LLM 测试稳定性和浏览器兼容表现，不影响本地核心功能的测试判定。

1.3 定义

同第一部分 1.3。

1.4 参考资料

同第一部分 1.4。

2 测试概要

测试工作于 2026-06-20 至 2026-06-25 按计划分阶段完成。各项测试均已覆盖计划中的主要输入、异常路径和验收准则。除 BearFRP 用户与代理测试中发现的余额边界和删除后一致性问题外，其余执行结果与测试设计中的预期输出一致，未发现影响课程演示主链路的阻塞级缺陷。

测试标识符	测试内容	计划内容	实际执行内容	差异说明
T-01	部署与启动测试	启动 Compose 全栈并检查健康状态	执行 Compose 启动、服务状态检查、health/online 冒烟测试	与计划一致。
T-02	LLM Wiki API 测试	测试 health、projects、files、search、graph、rescan	执行接口请求、错误 token、超大 body、测试资产统计	与计划一致。
T-03	OpenCode KB 权限测试	测试私有/公开目录、越权、禁用接口	执行 KB guard、server、Web UI 根节点和禁用接口测试	与计划一致。
T-04	BearFRP 用户与代理测试	测试注册、登录、充值、HTTP 代理创建和删	执行 HTTP 代理正常路径、重复子域名、余额边界、删除后	发现余额刚好用尽时代理被轮询停止、删除后列表

测试标识符	测试内容	计划内容	实际执行内容	差异说明
		除	一致性和异常路径测试	仍返回 deleted 代理两个问题。
T-05	frps 插件鉴权测试	测试 Login、NewProxy、Ping、CloseProxy	执行正确 token、旧 token、错子域名测试	与计划一致。
T-06	自动发布 sidecar 测试	测试 sidecar 自动发布 nginx	执行自动注册、充值、代理复用、配置重写和 URL 输出检查	与计划一致。
T-07	端到端访问测试	测试本地入口和 BearFRP 发布入口访问	执行浏览器访问、搜索、图谱和发布 URL 对比	与计划一致。
T-08	异常与安全测试	测试默认凭据、基础 XSS 样例、越权、请求体和子域名冲突	执行安全边界输入和错误路径测试	与计划一致。
T-09	桌面端知识库闭环测试	测试 Compile、Link 和本地 mask 发布页浏览	执行项目目录、Wiki 生成、链接报告、本地页面和前端提示检查	与计划一致。

3 测试结果及发现

3.1 T-01 部署与启动测试

检查项	输入或命令	实际结果	判定
Compose 配置解析	<code>docker compose config -q</code>	配置解析通过，无错误输出	通过
Compose 启动	<code>docker compose -p wikibridge ps --all</code>	llm-wiki、opencode、nginx、bearfrp、bearfrp-wikibridge-frpc 均处于运行状态	通过
nginx 健康检查	<code>curl http://127.0.0.1:18080/global/health</code>	返回 {"healthy":true,"version":"local"}	通过
LLM Wiki health	<code>curl http://127.0.0.1:18080/instance/llm-wiki/health</code>	返回 HTTP 200，服务状态为 running	通过
nginx Web 首页	<code>curl http://127.0.0.1:18080/</code>	返回 HTTP 200，HTML 响应大小为 3104 字节	通过
容器日志	<code>docker compose logs</code>	无持续重启和端口绑定失败	通过

发现：部署链路结构清晰，服务依赖关系正确；外部端口和默认凭据需要在公开部署前按环境调整。

3.2 T-02 LLM Wiki API 测试

检查项	输入或命令	实际结果	判定
API health	GET /api/v1/health	返回 status、version、authRequired 等字段	通过
项目列表	GET /api/v1/projects	返回 projects 数组和 current project	通过
文件列表	GET /api/v1/projects/default/files	返回文件树或文件数组	通过
文件内容	GET /api/v1/projects/default/files/content	返回指定文件内容	通过
搜索	POST /api/v1/projects/default/search	返回相关搜索结果，数量不超过 50	通过
图谱	GET /api/v1/projects/default/graph	返回 nodes 和 edges	通过
重扫	POST /api/v1/projects/default/sources/rescan	返回 rescan 接收或完成状态	通过
错误 token	Authorization: Bearer invalid-token	返回 401	通过
超大请求体	大于 1MB body	返回 400 或明确错误	通过
MCP 网络失败提示	桌面端未启动或连接拒绝	返回包含 Is the desktop app running? 的错误提示	通过
MCP 非 JSON 响应	网关返回 502 Bad Gateway 且 body 非 JSON	返回 non-JSON response (502) 和响应正文预览	通过
Review 标题规范化	Missing page:、Missing-Page:、缺失页面:、缺少页面:、大小写和多空格变体	归一为稳定标题键，重复评审项可去重	通过
Source Watch 配置规范化	md, pdf、中文逗号、换行和临时文件 glob	输入被拆分并归一化，文档类型允许，配置/二进制/临时文件拒绝	通过

过程记录：LLM Wiki API 和相关库级测试中，最初暴露出的不是接口完全不可用，而是异常信息和用户输入规范化不足。MCP 客户端遇到 ECONNREFUSED 或非 JSON 502 时，若只抛出底层错误，测试人员难以判断是桌面端未启动、网关异常还是响应格式异常；修正后错误消息增加桌面端运行提示、HTTP 状态码和正文预览。Review 标题和 Source Watch 配置也采用同样的规范化思路，将中英文前缀、大小写、全角/半角标点、多空格、逗号和换行输入统一为稳定内部表示，再执行去重或过滤判断。

发现：LLM Wiki API 边界定义明确，health 免鉴权，其余接口按 Token、方法和请求体限制处理；相关回归已覆盖错误可诊断性、评审标题规范化和资料扫描配置规范化。对话接口不作为本轮 API 验收项，相关能力由前端 chat-agent 测试覆盖。

3.3 T-03 OpenCode KB 权限测试

检查项	输入或命令	实际结果	判定
私有目录读取	data/users/default/README.md	读取成功	通过
私有目录写入	data/users/default/new.md	写入成功	通过
公开 Wiki 读取	data/wiki/README.md	读取成功	通过
公开 Wiki 写入	data/wiki/new.md	拒绝写入	通过
路径穿越	../package.json	拒绝访问	通过
其它用户目录	data/users/alice/README.md	拒绝访问	通过
软链接逃逸	私有目录软链接到项目外路径	realpath 后拒绝	通过
shell 接口	POST /session/:id/shell	返回 403	通过
pty/mcp/config	/pty、/mcp、/config	返回 403 或隐藏入口	通过

过程记录：KB 模式早期测试不能只验证“能读自己的知识库”，还必须确认不该发生的操作确实失败。依据 `opencode/doc/req.md` 中的必须失败清单，测试补充了读取 `../package.json`、读取其它用户目录、写公共 Wiki、使用 shell/终端和调用禁用接口等路径。权限策略收敛到服务端 KB Guard 和真实路径校验后，前端隐藏入口不再是唯一防线，越权请求即使绕过 UI 也会被拒绝。

发现：KB Guard 能够有效约束文件访问范围，Web UI 和服务端权限边界一致；公共 Wiki 写入、路径穿越、其它用户目录、软链接逃逸和 shell/pty/mcp/config 调用均作为失败样例纳入该测试项。

3.4 T-04 BearFRP 用户与代理测试

检查项	输入或命令	实际结果	判定
注册	POST /api/user/register	返回用户公开信息并建立 session	通过
登录	POST /api/user/login	返回用户公开信息	通过
充值	POST /api/user/recharge	余额增加	通过
HTTP 代理	POST /api/proxies	返回 subdomain URL、frpc_config 和 scripts	通过
HTTP 高级配置	HTTP basic auth、locations、host header rewrite	配置写入 frpc HTTP 代理配置	通过
名称重复	重复提交同名代理	返回 400	通过
余额不足	traffic_mb 超过余额	返回 400	通过
非法子域名	空值、含点号或下划线	返回 400	通过

检查项	输入或命令	实际结果	判定
删除代理	DELETE /api/proxies/{id}	状态删除，子域名不可继续访问	通过
余额刚好用尽时创建连接	用户余额为 50MB，创建 traffic_mb=50 的 HTTP 连接	API 创建成功，frpc 尝试建立连接，但后端轮询后将代理置为 stopped_by_admin	不通过
删除连接后用户统计与列表一致性	创建连接后删除，再分别请求 /api/user/me 和 /api/proxies	connection_count=0，但 /api/proxies 仍返回 status=deleted 的连接	不通过

过程记录：BearFRP 控制面测试在正常创建、重复名称、余额不足和非法子域名之外，继续补充了两个边界失败样例。第一类是余额刚好等于连接流量额度时，创建接口按余额校验放行，但轮询阶段又把代理停为 stopped_by_admin，说明创建校验、运行态扣费或轮询停用规则之间存在边界不一致。第二类是删除连接后，用户统计已按删除结果更新为 connection_count=0，但代理列表仍返回 status=deleted 记录，说明列表接口和统计口径不一致，容易造成用户误以为已删除连接仍存在。

发现：代理创建流程已覆盖余额、名称、连接数、HTTP 子域名、HTTP 高级配置和删除分支，错误提示能够定位主要失败原因；但余额刚好用尽和删除后列表一致性两个用例未通过，需要作为 BearFRP 控制面缺陷继续修正并回归。

3.5 T-05 frps 插件鉴权测试

检查项	输入或命令	实际结果	判定
Login 正常	正确 token	允许连接	通过
Login 异常	旧 token 或伪造 token	拒绝连接	通过
NewProxy 正常	匹配代理名和 HTTP 子域名	允许创建代理	通过
NewProxy 错子域名	subdomain 不一致	拒绝	通过
Ping token_version	旧版本 token	拒绝或要求重连	通过
CloseProxy	关闭代理	在线状态更新	通过

发现：frps 插件能防止旧 token、错子域名和已删除代理继续占用 HTTP 发布入口。

3.6 T-06 自动发布 sidecar 测试

检查项	输入或命令	实际结果	判定
API 等待	sidecar 启动	BearFRP API ready 后继续	通过
自动注册/登录	默认演示用户	用户存在则登录，不存在则注册	通过
自动充值	余额不足	自动充值至满足代理创建	通过

检查项	输入或命令	实际结果	判定
代理创建	wikibridge 代理	创建或更新成功；TCP 模式使用 tcp_ports.mode=auto	通过
在线代理查询	curl http://127.0.0.1:8000/api/show/online	返回 wikibridge TCP 代理，远程端口 52600，本地端口 80，is_online=true	通过
配置重写	frpc.toml	serverAddr/serverPort 指向 Compose 内 BearFRP	通过
发布 URL 输出	sidecar 日志或 online 接口	输出 WikiBridge BearFRP URL： http://localhost:52600/	通过
同名异类代理	已存在不同 proxy_type	报错并停止，避免误复用	通过

发现：sidecar 能够支持课程演示的一键发布路径，并对同名异类代理给出明确失败信息。

3.7 T-07 端到端访问测试

检查项	输入或命令	实际结果	判定
本地入口	http://127.0.0.1:18080/	OpenCode KB 页面可访问，返回 HTTP 200	通过
KB meta	页面 HTML	包含 opencode-kb-mode	通过
项目列表	页面或 API	可显示项目	通过
文件浏览	点击 Wiki 文件	页面内容可显示	通过
搜索	输入 example	返回相关结果	通过
图谱	打开 graph	节点和边显示	通过
BearFRP 发布入口健康检查	curl http://127.0.0.1:52600/global/health	返回 { "healthy":true,"version":"local" }	通过
BearFRP 发布首页	curl http://127.0.0.1:52600/	返回 HTTP 200，HTML 响应大小为 3104 字节，与 nginx 入口一致	通过

发现：本地入口和 BearFRP 发布入口的主要页面、健康检查和知识库访问路径一致，满足课程演示闭环要求。

3.8 T-08 异常与安全测试

检查项	输入或命令	实际结果	判定
默认凭据检查	<code>.env</code> 默认值	对外部署前提示修改	通过
错误 token	<code>Bearer invalid-token</code>	返回 401	通过
基础 XSS 样例	<code><script>alert(1)</script></code>	未触发弹窗或脚本执行	通过
路径穿越	<code>../package.json</code>	拒绝访问	通过
超大 API body	> 1MB	返回错误	通过
超大网关 body	> 50MB	nginx 拒绝	通过
子域名冲突	已占用子域名	创建失败且不覆盖原代理	通过
未认证用户接口	<code>GET /api/user/me</code> ，不带登录 Cookie	返回 HTTP 401，提示 <code>user login required</code>	通过
未认证代理脚本接口	<code>GET /api/proxies/999999/scripts</code> ，不带登录 Cookie	返回 HTTP 401，优先拒绝未登录访问	通过
未认证代理创建	<code>POST /api/proxies</code> ，不带登录 Cookie	返回 HTTP 401，拒绝创建代理	通过
用户名长度下界	注册用户名 <code>ab</code> 、密码 <code>123</code>	返回 HTTP 400，提示用户名需为 3-32 位字母、数字或下划线	通过
空用户名与空密码	注册用户名为空、密码为空	返回 HTTP 400，提示用户名格式错误	通过
错误用户密码	用户 <code>wikibridge</code> 使用错误密码登录	返回 HTTP 401，提示用户名或密码错误	通过
错误管理员密码	管理员 <code>admin</code> 使用错误密码登录	返回 HTTP 401，提示 <code>invalid credentials</code>	通过
未知后端路径	<code>GET http://127.0.0.1:8000/__not_found__</code>	返回 HTTP 404，提示 Not Found	通过

过程记录：本项测试中的多数组合输入本身就是预期失败样例，用于验证系统是否按安全边界失败。例如错误 token、未认证接口、非法用户名、错误密码、子域名冲突和未知路径均应返回明确的 4xx 结果，而不是静默成功、覆盖已有资源或泄露内部信息。测试执行后，上述失败被规范化为可解释的错误响应，正常登录、充值、代理创建和知识库访问流程不受影响。

发现：系统主要安全边界符合预期；基础脚本注入样例未触发执行。公开部署前仍需替换默认密码和 token，并建议后续补充更系统的 Markdown 安全回归用例。

3.9 T-09 桌面端知识库闭环测试

检查项	输入或命令	实际结果	判定
项目初始化	创建桌面端知识库项目	自动建立 <code>raw/</code> 和 <code>wiki/</code> 子目录	通过
Compile 构建	无素材、文本、图片和二进制素材	进入 <code>Building</code> 状态；无素材生成 <code>wiki/README.md</code> ；文本、图片、二进制素材分别生成 Markdown 页面、图片引用页和元数据说明页	通过
Link 检查	已生成 Wiki 页面和 <code>[[...]]</code> 链接	进入 <code>Linking</code> 状态；识别反链和缺失链接；生成 <code>wiki/_link_report.md</code> ；支持标题、文件名、去数字前缀和 slug 候选匹配	通过
本地 mask 发布页	<code>/api/project</code> 、 <code>/</code> 、 <code>/wiki/<page></code> 、 <code>/raw/<asset></code>	保留项目 JSON 输出；首页展示 Wiki 与状态摘要；单篇 Wiki 可浏览；原始图片按 MIME 类型返回	通过
页面渲染	Wiki 链接、标题、段落、代码块、引用、列表、图片	<code>[[...]]</code> 转换为可点击页面链接，基础 Markdown 正确渲染为 HTML	通过
前端成功提示	Compile 和 Link 成功	分别提示 <code>wiki 已构建</code> 、 <code>链接检查已完成</code>	通过
多来源页面保留	两个素材生成同一 Wiki 页面后删除其中一个素材	页面保留在磁盘， <code>sources</code> 仅移除被删除来源	通过
错误透传	Wiki 相关错误	前端直接展示错误信息，便于定位	通过

过程记录：桌面端闭环测试除验证最终页面可浏览外，还覆盖了多来源页面的数据保留问题。初始风险是第二次摄入同一主题页面时，后一次 `sources` 可能覆盖第一次来源；用户删除后一次素材时，共享页面会被当成只属于该素材而误删。回归中将来源字段合并为并集，并把删除决策拆成 `skip`、`keep`、`delete` 三类：待删来源不在页面来源中则跳过，仍有其它来源则保留并重写 `frontmatter`，唯一来源才删除。

发现：桌面端知识库闭环已从“仅更新状态”推进为“生成 Wiki 页面、生成链接检查报告、通过本地 mask 服务浏览 Wiki”的可演示实现；多来源页面删除场景已纳入回归，防止资料重构过程中发生误删。

4 对软件功能的结论

4.1 功能 F-01：部署与统一入口

4.1.1 能力

系统能够通过 Docker Compose 启动多服务栈，nginx 作为统一入口转发到 OpenCode KB Web 服务，并通过 OpenCode 桥接 LLM Wiki API。该功能满足课程演示中“打开一个入口访问知识库”的要求。

4.1.2 限制

该能力依赖 Docker 环境、端口可用性和主机资源。对外部署时还需配置 HTTPS、DNS、防火墙和安全凭据。

4.2 功能 F-02: LLM Wiki 知识库服务

4.2.1 能力

LLM Wiki 能提供项目列表、文件列表、文件内容、评审记录、搜索、图谱和素材重扫接口，并具备 Token 鉴权、请求体限制、文件大小限制、限流和并发保护。

4.2.2 限制

real-LLM 相关能力依赖外部模型服务和 API Key；大文件和大量文件项目可能需要进一步优化进度提示和分块处理。

4.3 功能 F-03: OpenCode KB 权限隔离

4.3.1 能力

OpenCode KB 模式能将文件访问限制在用户私有目录和公开 Wiki 目录内；私有目录可读写，公开 Wiki 只读，项目外路径、其它用户目录和软链接逃逸均被拒绝。

4.3.2 限制

当前用户身份主要由环境变量和部署配置控制，生产环境可进一步接入登录会话、审计日志和管理员维护流程。

4.4 功能 F-04: BearFRP 发布控制面

4.4.1 能力

BearFRP 能完成用户注册登录、流量充值、HTTP/TCP 代理创建、脚本生成、Token 轮换、代理删除和子域名或远程端口释放。当前测试中 TCP 发布入口已覆盖发布闭环，HTTP 子域名代理也已覆盖控制面核心路径。

4.4.2 限制

HTTP 子域名模式依赖 subdomain host、vhost 端口和防火墙配置。控制面仍存在两个需修复的边界问题：余额刚好等于连接额度时，创建接口放行但轮询后代理被停为 `stopped_by_admin`；删除代理后用户统计已更新，但代理列表仍返回 `status=deleted` 记录。

4.5 功能 F-05: 自动发布 sidecar

4.5.1 能力

sidecar 能自动等待 BearFRP API、注册或登录演示用户、补足余额、创建或更新发布代理、改写 frpc 配置，并输出可访问的 BearFRP 发布 URL，满足一键演示发布需求。

4.5.2 限制

sidecar 依赖 GitHub frp release 下载 frpc；离线环境下需要提前提供 frpc 二进制或镜像缓存。

4.6 功能 F-06: 桌面端知识库闭环

4.6.1 能力

桌面端能够在创建项目时准备 `raw/` 和 `wiki/` 目录，通过 `Compile` 将文本、图片和暂不解析正文的二进制素材转换为可浏览的 Wiki 页面，通过 `Link` 生成 `wiki/_link_report.md` 并统计反链、缺失链接。本地 `mask` 发布页能够展示 Wiki 首页、页面目录、构建状态、链接状态、素材数量、单篇 Wiki 文档和原始图片素材，并将 `[[...]]` 链接转换为可点击页面链接。

4.6.2 限制

当前 `Compile` 对二进制素材以元数据说明页为主，不解析正文内容；复杂 `Markdown` 扩展语法和大规模素材项目仍需后续专项测试。

5 分析摘要

5.1 能力

本轮测试结果证实 WikiBridge 已具备以下能力：

- 1. 能以 `Docker Compose` 方式部署为完整多服务系统。
- 2. 能通过 `nginx` 单入口访问知识库消费端。
- 3. 能通过 `OpenCode KB` 模式隔离用户私有知识库和公开 Wiki。
- 4. 能通过 `LLM Wiki API` 提供项目、文件、搜索和图谱等知识库能力。
- 5. 能通过 `BearFRP` 创建 `HTTP/TCP` 发布代理并生成 `frpc` 配置和脚本。
- 6. 能通过 `sidecar` 自动发布 `nginx` 网关，形成 `BearFRP` 发布访问闭环。
- 7. 能在当前测试环境中通过 `TCP` 发布入口 `http://localhost:52600/` 访问与 `nginx` 单入口一致的 Web 首页。
- 8. 能通过桌面端完成本地素材到 Wiki 页面、链接检查报告和本地 `mask` 浏览页的闭环。
- 9. 能对错误 `token`、路径越权、子域名冲突、未认证访问、非法用户名、错误凭据、超大请求体和基础脚本注入样例等风险进行防护。

5.2 缺陷和限制

第 3 章各测试项中记录的已关闭过程性问题已纳入回归检查。本节保留当前版本仍需在部署、依赖、体验或控制面一致性层面继续处理的缺陷和限制。

编号	缺陷或限制	影响	严重性
L-01	默认配置中存在演示用默认密码或空密码入口	公开部署时存在安全风险	高
L-02	real-LLM 测试依赖外部服务	网络或模型服务异常会影响测试稳定性	中
L-03	HTTP 子域名模式依赖 subdomain host、DNS 或本地域名解析配置	未配置域名解析时只能使用 host/端口或本地 hosts 方式演示	中
L-04	大文件和多文件摄入可能耗时较长	用户体验受影响	中
L-05	sidecar 首次运行可能需要下载 frpc	离线环境下启动失败	中
L-06	桌面端 <code>Compile</code> 对二进制素材暂不解析正文	二进制文件只能生成元数据说明页，无法形成全文知识内容	低

编号	缺陷或限制	影响	严重性
L-07	BearFRP 余额刚好用尽时创建连接会被轮询停用	用户认为创建成功，但代理随后进入 <code>stopped_by_admin</code> ，发布链路不稳定	高
L-08	BearFRP 删除连接后用户统计与代理列表口径不一致	<code>connection_count=0</code> 但列表仍显示 <code>status=deleted</code> 连接，影响用户判断和后续自动化校验	中

5.3 建议

编号	建议	紧迫程度	预计工作量	负责人
S-01	对外部署前强制修改 <code>OPENCODE_SERVER_PASSWORD</code> 、BearFRP 管理密码、frps token 和 <code>LLM_WIKI_TOKEN</code>	高	0.5 人日	部署负责人
S-02	增加 <code>scripts/verify-compose.sh</code> ，固化 health、projects、search、KB meta、BearFRP online 检查	高	1 人日	测试负责人
S-03	为 BearFRP 后端补充轻量 pytest 集成测试	中	2 人日	后端负责人
S-04	将 OpenCode KB guard/server 测试纳入后续自动化回归	中	1 人日	前端/服务端负责人
S-05	为 sidecar 增加 mock API 测试和离线 frpc 缓存方案	中	1.5 人日	部署负责人
S-06	大文件摄入增加进度、取消和重试提示	低	2 人日	LLM Wiki 负责人
S-07	为桌面端 Compile/Link 和本地 mask 发布页补充自动化回归样例	中	1.5 人日	桌面端负责人
S-08	修正 BearFRP 余额边界规则，使创建校验、运行态扣费和轮询停用条件一致	高	1 人日	BearFRP 后端负责人
S-09	统一 BearFRP 删除代理后的统计与列表口径，明确 <code>/api/proxies</code> 是否默认过滤 deleted 记录	中	0.5 人日	BearFRP 后端负责人

5.4 评价

从测试计划覆盖范围和执行结果看，WikiBridge 已达到课程设计预定目标：系统能够完成本地知识库服务、受控知识库消费端、桌面端知识库生成与本地浏览、统一 Web 入口和 BearFRP 发布的核心闭环。系统可以用于课程设计演示和验收。公开部署前需要完成默认凭据替换、HTTPS 配置和 HTTP vhost/DNS 或 TCP 端口开放检查，并修正 BearFRP 控制面的余额边界和删除后一致性问题。

6 测试资源消耗

资源类型	数量或时间	说明
测试人员	2 人	1 人执行接口和部署测试，1 人执行浏览器和安全边界测试。

资源类型	数量或时间	说明
开发协助人员	1 人	负责定位源码、解释业务逻辑和修正阻塞问题。
测试主机	1 台	Linux + Docker Compose 环境。
浏览器客户端	3 类	Chrome、Edge、Firefox/Safari。
机时消耗	约 6 小时	包括构建、启动、接口测试、浏览器测试和报告整理。
自动化测试时间	约 1.5 小时	包括 LLM Wiki、OpenCode KB、BearFRP 后端语法检查和相关构建。
人工测试时间	约 4.5 小时	包括 E2E、异常输入、安全边界、桌面端知识库闭环和截图记录。

附录 A 测试命令清单

```
# 启动完整集成栈
cp .env.example .env
docker compose up --build -d

# Compose smoke test
curl http://localhost/instance/llm-wiki/health
curl http://localhost/instance/llm-wiki/projects
curl -X POST -H "content-type: application/json" \
  -d '{"query":"example"}' \
  http://localhost/instance/llm-wiki/projects/default/search
curl http://localhost/ | grep 'opencode-kb-mode'
curl http://localhost:8000/api/show/online
curl http://127.0.0.1:18080/global/health
curl http://127.0.0.1:18080/instance/llm-wiki/health
curl http://127.0.0.1:52600/global/health
curl http://127.0.0.1:52600/

# BearFRP 后端语法编译
cd bearfrp
python -m compileall backend

# BearFRP 桌面端前端构建
cd bearfrp/desktop-frp
npm ci
npm run build

# LLM Wiki 前端和库测试
cd llm_wiki
npm ci
npm run build
npm run test:mocks

# LLM Wiki real-LLM 测试, 需配置 API Key
npm run test:llm

# LLM Wiki MCP server 测试
npm run mcp:test

# OpenCode KB 模式测试
cd opencode
```

```
bun install --frozen-lockfile
cd packages/opencode
bun test test/kb/guard.test.ts test/kb/server.test.ts
bun run typecheck
```

附录 B 重点回归用例清单

编号	回归用例	预期结果
REG-01	Compose 启动后打开 nginx 单入口	OpenCode KB Web UI 可访问。
REG-02	请求 <code>/instance/llm-wiki/health</code>	返回健康状态。
REG-03	请求 <code>/instance/llm-wiki/projects</code>	返回项目列表。
REG-04	执行知识库搜索	返回可跳转结果。
REG-05	查看知识图谱	返回节点和边。
REG-06	KB 模式下写私有目录	写入成功。
REG-07	KB 模式下写公开 Wiki	写入被拒绝。
REG-08	KB 模式下读取项目外文件	访问被拒绝。
REG-09	KB 模式下访问 shell、pty、mcp、config	返回 403 或隐藏入口。
REG-10	BearFRP 注册新用户并登录	返回用户信息。
REG-11	充值后创建 HTTP 代理	返回 subdomain URL 和脚本，重复子域名校验生效。
REG-12	创建重复子域名 HTTP 代理	返回 400。
REG-13	创建非法子域名 HTTP 代理	返回 400。
REG-14	轮换 frpc token 后使用旧脚本	连接被拒绝。
REG-15	sidecar 自动发布 <code>wikibridge</code> 代理	日志或在线代理接口输出 BearFRP 发布 URL。
REG-16	访问 BearFRP 发布 URL	主要页面与 nginx 单入口一致。

编号	回归用例	预期结果
REG-17	删除代理后访问 BearFRP 发布 URL	访问失败，子域名或远程端口释放。
REG-18	创建桌面端知识库项目	自动建立 <code>raw/</code> 和 <code>wiki/</code> 子目录。
REG-19	桌面端执行 Compile	生成 Wiki 页面；无素材时生成提示性 <code>wiki/README.md</code> 。
REG-20	桌面端执行 Link	生成 <code>wiki/_link_report.md</code> ，包含反链统计和缺失链接记录。
REG-21	访问本地 mask 发布页 <code>/wiki/<page></code>	可浏览 Wiki 首页和单篇 Wiki 文档。
REG-22	访问本地 mask 发布页 <code>/raw/<asset></code>	图片素材按对应 MIME 类型返回。
REG-23	检查桌面端 Compile/Link 前端提示	分别显示 <code>Wiki 已构建</code> 和 <code>链接检查已完成</code> 。
REG-24	多来源页面重复摄入后删除其中一个来源	页面保留在磁盘， <code>sources</code> 仅移除被删除来源，未关联页面不被误删。
REG-25	评审标题包含中英文缺失页/重复页前缀、大小写和多空格	标题归一后可稳定去重，非前缀正文冒号被保留。
REG-26	Source Watch 输入包含英文逗号、中文逗号、换行和临时文件 glob	配置被拆分并归一化，文档类型被允许，配置/二进制/临时文件被拒绝。
REG-27	KB 模式下尝试路径穿越、公共 Wiki 写入、其它用户目录和 shell/pty/mcp/config 接口	所有越权或禁用操作均失败，私有目录和公共 Wiki 只读访问不受影响。
REG-28	MCP/API 客户端遇到网络拒绝或非 JSON 错误响应	错误消息包含桌面端运行提示或 HTTP 状态码和响应正文预览。
REG-29	用户余额刚好等于 <code>traffic_mb</code> 时创建 HTTP 连接	创建、frpc 连接和后端轮询状态保持一致，不应在无超额使用前被置为 <code>stopped_by_admin</code> 。
REG-30	删除 BearFRP 连接后分别查询 <code>/api/user/me</code> 和 <code>/api/proxies</code>	用户统计与列表口径一致；若 <code>connection_count=0</code> ，默认代理列表不应继续展示已删除连接，或需明确通过筛选参数查询历史记录。